

Route 4: MATLAB 修复前后 vs Python 对照报告

1. 报告目的

本报告专门回答一个具体问题：

为什么在同样的 `route4 / phase-insensitive primal` 逻辑下，
MATLAB 原脚本会给出 $H_{\min} \approx 1.854693$ bits，
而 Python 对应实现只给出 $H_{\min} \approx 0.161058$ bits？

结论先行：

- 这不是“MATLAB 和 Python 用不同语言所以有小误差”的问题；
- 也不是“换个求解器所以数值稍微不同”的问题；
- 而是 MATLAB 原脚本在构造输入态 `rho_diag` 时已经引入了严重数值错误，导致它实际送进求解器的不是和 Python 同一个优化实例；
- 后续 SDPT3 又在这个病态实例上返回了一个表面上 `solved`、但实际上不可信的结果。

2. 对比对象

本次比较的对象是：

- 原 MATLAB 脚本：
`src/matlab/guessprobprimal_phaseinsensitive.m`
- Python `route4` 实现：
`src/python/qrng_routes/route4/phaseinsensitive.py`

比较时统一使用当前 MATLAB 文件里的参数：

- `selected_mu_list = [100, 120, 140]`
- `q_selected = [1/4, 1/4, 1/2]`
- `M = 280`
- `N = 4`
- `shift = 0`
- `exact` 逻辑，即不使用 `prob_floor`

3. 直接结论

在上述同参数下，Python 的 exact primal / dual 结果为：

- primal:
 $p_guess \approx 0.8943688585$
 $H_min \approx 0.1610581399$
- dual:
 $p_guess \approx 0.8943688978$
 $H_min \approx 0.1610580764$

而用户给出的 MATLAB 运行结果是：

- $p_guess \approx 0.276491$
- $H_min \approx 1.854693$

这两组结果差异非常大，不可能仅由“求解器不同”解释。

真正原因是：

1. MATLAB 原脚本构造出的 `rho_diag` 不是合法的、归一化的输入态对角分布；
2. 这个错误实例会把 primal / dual 问题推向病态；
3. SDPT3 在该病态实例上没有给出可信最优解，但 CVX 仍把状态显示成了 `Solved`。

4. 哪些部分其实是一致的

在正式分析差异前，需要先排除两个最容易误判的来源。

4.1 Probability.mat 的读法是一致的

对当前参数：

- `full_mu = [0,20,40,60,80,100,120,140,160]`
- `selected_mu_list = [100,120,140]`
- `shift = 0`

MATLAB 和 Python 读到的其实是同样三行数据，对应：

- 100
- 120
- 140

因此这次差异不是由“索引错位”引起的。

4.2 N = 4 时的 coarse-graining 也是一致的

当前 MATLAB 文件中：

```
N = 4
block_size = round(256 / 4) = 64
```

而 Python route4 修复版在 N = 4 时的等覆盖边界是：

```
[0, 64, 128, 192, 256]
```

所以这时两边的 256 -> 4 分箱完全一样。

实际比对后的 $p(x,y)$ 也一致：

输入态	4 个 coarse-grained 输出概率
$\mu = 100$	[2e-08, 0.53859586, 0.461395615, 8.505e-06]
$\mu = 120$	[0, 0.06736786, 0.93062086, 0.00201128]
$\mu = 140$	[0, 0.001395395, 0.928520685, 0.07008392]

因此：

- Probability.mat 不是差异来源；
- N = 4 的 coarse-graining 也不是差异来源。

5. 真正主因：MATLAB 原脚本把 rho_diag 构坏了

5.1 原 MATLAB 脚本的问题写法

原 MATLAB 脚本使用：

```
coeff(n+1) = exp(-abs(alpha)^2 / 2) * (alpha^n) / sqrt(factorial(n));
rho(i, :, :) = coeff * coeff';
rho_diag(i, :) = diag(rho_i_2d);
```

这在低光子数下看起来没问题，但对 $M = 280$ 来说有一个严重数值问题：

- 双精度下，`factorial(n)` 从大约 $n \geq 171$ 开始会溢出到 `Inf` ；
- 一旦分母变成 `Inf` ，对应的 `coeff(n+1)` 就直接掉成 `0` ；
- 于是高光子数尾部被错误截断；
- 更关键的是，原脚本没有在截断后重新归一化。

于是 `rho_diag` 不再是 `trace = 1` 的合法输入态。

5.2 这个问题有多严重

在当前参数下，原 MATLAB 这一写法构造出的三组输入态对角元，其迹约为：

mu	原 MATLAB 构造后的 trace(rho)
100	0.99999999993
120	0.99999310630
140	0.99388344361

其中 $\mu = 140$ 时已经丢掉了约：

$$1 - 0.99388344361 \approx 0.00611655639$$

这不是一个可以忽略的偏差，因为后面的 `primal` 约束是 `exact equality`：

$$\text{rho_diag}(x,:) * M_total(:,y) == p(x,y)$$

右边的 $p(x,y)$ 来自实验统计，默认对应的是一个合法归一化输入态；
左边却用了一个已经丢失概率质量的“坏 ρ ”。

这样一来，优化问题本身就被扭曲了。

6. 关键证据：把 MATLAB 的坏 `rho_diag` 喂回 Python，会发生什么

为了隔离根因，做了两个非常重要的反向实验。

6.1 实验 A：把 MATLAB 那种“溢出后未归一化”的 `rho_diag` 直接喂给 Python

保持其他所有内容不变，只把 Python 里的 `rho_diag` 换成 MATLAB 原脚本那种构造结果。

得到：

- primal: infeasible
- dual: unbounded

这说明：

- MATLAB 原脚本实际构造出来的输入态，会把 exact primal / dual 问题直接推向病态；
- 它并不是在解和 Python 一样的那个正常实例。

6.2 实验 B：尾部仍被截断，但补上归一化

如果保留 MATLAB 那种尾部截断，只是额外做：

```
rho_diag <- rho_diag / trace(rho_diag)
```

那么 Python 结果立刻回到：

- primal: $H_{\min} \approx 0.1612967051$
- dual: $H_{\min} \approx 0.1612967020$

它已经非常接近稳定版 Python 的：

- $H_{\min} \approx 0.161058$

这说明：

- 真正致命的不是“高光子数尾部略有截断”本身；
- 真正致命的是“截断之后没有重新归一化”。

7. 为什么 MATLAB 还会返回一个看起来很好的 1.854693 bits

这就涉及第二层问题：求解器行为。

用户贴出的 SDPT3 日志里已经出现了多个危险信号：

- checkdepconstr: AAt is not pos. def.
- lack of progress in infeas
- actual relative gap = -1.00e+00
- norm(y) = 1.4e+09

这些都不是“健康、稳定收敛”的迹象。

更准确地说，它们说明：

- 约束系统存在严重病态；
- 路径跟踪过程在 infeasibility 附近卡住；
- 当前返回点很可能只是一个数值上勉强停下来的点，而不是可信的最优解。

但是 CVX 在这种情况下仍可能把状态显示成：

Status: Solved

这就容易造成误判，好像 `cvx_optval = 0.276491` 是一个可靠的最大猜测概率。

实际上，这个值与稳定版 Python 的结果相比相差太大，已经不能被当作同一物理问题的可信答案。

8. 为什么这不可能只是“换求解器带来的误差”

在 Python 里，对同一个 `exact N = 4` 实例分别尝试了多个求解器。

8.1 primal

求解器	状态	H_min
MOSEK	optimal	0.1610581399
CLARABEL	optimal	0.1610577995
SCS	optimal	0.1604009061

8.2 dual

求解器	状态	H_min
MOSEK	optimal	0.1610580764

求解器	状态	H_min
CLARABEL	optimal	0.1610580775
SCS	optimal	0.1609734219
SCIPY	unbounded	null
OSQP	user_limit	null

这说明：

- 对“健康”的 exact N = 4 实例，合适求解器之间的结果只会有很小差异；
- 不可能从 0.161 突然跳到 1.855 ；
- 因此 MATLAB 当前结果的巨大偏差不是“语言差异”或“solver 小误差”，而是实例本身已经被构坏了。

9. MATLAB 原代码和执行结果具体存在哪些问题

这里把问题分成“代码层面”和“执行结果层面”两部分列清楚。

9.1 原 MATLAB 代码层面的问题

1. 输入态构造使用 factorial(n) ，在 M = 280 下会溢出。
2. 高光子数尾部被截断后，没有重新归一化。
3. 于是 rho_diag 不再是 trace = 1 的合法密度矩阵对角元。
4. 后续 exact equality 约束实际上在匹配一个“坏输入态”和“正常实验统计”。
5. 默认使用 SDPT3 时，没有对病态求解状态做额外筛查。

9.2 原 MATLAB 执行结果层面的问题

1. SDPT3 日志已经显示出明显的病态信号。
2. 尽管如此，CVX 仍显示 Status: Solved ，容易误导。
3. 最终给出的 p_guess ≈ 0.276491 、 H_min ≈ 1.854693 与稳定版 primal / dual 结果严重不一致。
4. 因此这组结果不应被当作 route4 在当前参数下的可信认证值。

10. 本次对 MATLAB 脚本做了什么修复

当前已将 src/matlab/guessprobprimal_phaseinsensitive.m 改为稳定版，核心修改有两点。

10.1 用对数域构造 rho_diag

把原来的：

```
exp(-abs(alpha)^2 / 2) * (alpha^n) / sqrt(factorial(n))
```

改成直接在对数域构造 Poisson 对角分布：

```
log_probs = -mu + n*log(mu) - gammaln(n+1)
diag_i = exp(log_probs)
```

这样可以避免 factorial(n) 溢出。

10.2 显式归一化

每个输入态构造完后，显式执行：

```
rho_diag(i,:) = diag_i / sum(diag_i)
```

从而保证每个输入态都是 trace = 1 的合法对角密度矩阵。

10.3 优先尝试 MOSEK

脚本现在会优先尝试：

```
cvx_solver mosek
```

如果当前 MATLAB / CVX 环境中没有安装 MOSEK，则退回默认求解器，并给出 warning。

这一步不是改变物理模型，而是尽量避免再出现 SDPT3 在病态点上“表面 solved、实际不可信”的情况。

11. 修复后应该怎样理解 MATLAB 和 Python 的关系

修复后的 MATLAB primal 脚本，与 Python route4 的关系应当这样理解：

- 优化逻辑仍然是同一套 primal 逻辑；
- 读入的 Probability.mat 仍然是同一份实验统计；
- 只是 MATLAB 端原来有严重的输入态数值构造问题，现在已经改为稳定版；

- 修复后，若求解器本身也足够可靠，MATLAB 结果应当回到与 Python ≈ 0.161 同一数量级，而不会再出现 ≈ 1.855 这种假高熵结果。

12. 最终结论

最后把结论压缩成三句话。

第一，原 MATLAB 和 Python 的巨大差异，不是因为它们在解“同一个健康实例”时出现了普通数值误差，而是因为 MATLAB 原脚本把输入态 `rho_diag` 构坏了。

第二，原 MATLAB 的 $H_{\min} \approx 1.854693$ bits 不应被视为可信结果。它更像是“坏输入态 + 病态约束 + SDPT3 数值不稳”共同作用下产生的伪结果。

第三，修复 `rho_diag` 的稳定构造与归一化之后，MATLAB 应当回到与 Python stable route4 一致的结果区域，也就是当前参数下 $H_{\min} \approx 0.161$ 的量级，而不是 1.8 bit 以上。